Search Reset

TABLE OF CONTENTS

MICRONAUT SCOPE TYPES

Learn about the available scopes: Singleton, Prototype, Request, Refreshable, Context...

Authors: Sergio del Amo

Micronaut Version: 5.0.0

1. Introduction

Micronaut features an extensible bean scoping mechanism based on **JSR-330**.

2. What you will need

To complete this guide, you will need the following:

- Some time on your hands
- A decent text editor or IDE (e.g. **IntelliJ IDEA**)
- JDK 21 or greater installed with `JAVA_HOME` **configured appropriately**

3. Solution

We recommend that you follow the instructions in the next sections and create the application step by step. However, you can go right to the **completed example**.

- **Download** and unzip the source

4. Writing the Application

Create an application using the **Micronaut Command Line Interface** or with **Micronaut Launch**.

```
mn create-app example.micronaut.micronautguide \
    --features=management,validation \
    --build=gradle \
    --lang=groovy \
    --test=spock
```

Copy

If you don't specify the `--build` argument, Gradle with the Kotlin DSL is used as the build tool.
If you don't specify the `--lang` argument, Java is used as the language.
If you don't specify the `--test` argument, JUnit is used for Java and Kotlin, and Spock is used for Groovy.

The previous command creates a Micronaut application with the default package `example.micronaut` in a directory named `micronautguide`.

If you use Micronaut Launch, select Micronaut Application as application type and add `management`, and `validation` features.

If you have an existing Micronaut application and want to add the functionality described here, you can view the dependency and configuration changes from the specified features, and apply those changes to your application.

Diff

Setup IntelliJ IDEA to develop Micronaut Applications.

5. Scenario

We use the following scenario to talk about the different types of scopes.

The following `@Controller` injects two collaborators.

src/main/groovy/example/micronaut/singleton/RobotController.groovy

```
package example.micronaut.singleton

import io.micronaut.http.annotation.Controller
import io.micronaut.http.annotation.Get

@Controller('/singleton') (1)

class RobotController {

    private final RobotFather father
    private final RobotMother mother

    RobotController(RobotFather father,  (2)
                    RobotMother mother) { (3)
        this.father = father
        this.mother = mother
    }

    @Get (4)
    List<String> children() {
        [
                father.child().serialNumber,
                mother.child().serialNumber
        ]
    }
}
```

Copy

1. The class is defined as a controller with the @Controller annotation mapped to the path `/`.
2. Use constructor injection to inject a bean of type `RobotFather`.
3. Use constructor injection to inject a bean of type `RobotMother`.
4. The @Get annotation maps the `children` method to an HTTP GET request on `/`.

Each collaborator has an injection point for a bean of type `Robot`.

src/main/groovy/example/micronaut/singleton/RobotFather.groovy

Copy

```groovy
package example.micronaut.singleton

import jakarta.inject.Singleton

@Singleton (1)
class RobotFather {
    private final Robot robot

    RobotFather(Robot robot) { (2)
        this.robot = robot
    }

    Robot child() {
        robot
    }
}
```

(1) Use `jakarta.inject.Singleton` to designate a class as a singleton.
(2) Use constructor injection to inject a bean of type `Robot`.

src/main/groovy/example/micronaut/singleton/RobotMother.groovy

```groovy
package example.micronaut.singleton

import jakarta.inject.Singleton

@Singleton (1)
class RobotMother {
    private final Robot robot

    RobotMother(Robot robot) { (2)
        this.robot = robot
    }

    Robot child() {
        robot
    }
}
```

(1) Use `jakarta.inject.Singleton` to designate a class as a singleton.
(2) Use constructor injection to inject a bean of type `Robot`.

Let's discuss how the application behaves depending on the scope used for the bean of type `Robot`.

6. Singleton

Singleton scope indicates only one instance of the bean will exist.

To define a singleton, annotate a class with `jakarta.inject.Singleton` at the class level.

The following class creates a unique identifier in the constructor. This identifier allows us to identify how many `Robot` instances are used.

src/main/groovy/example/micronaut/singleton/Robot.groovy

```groovy
package example.micronaut.singleton

import jakarta.inject.Singleton

import java.util.UUID

@Singleton (1)
class Robot {
    private final String serialNumber = UUID.randomUUID().toString()

    String getSerialNumber() {
        serialNumber
    }
}
```

(1) Use `jakarta.inject.Singleton` to designate a class as a singleton.

6.1. Singleton Test

The following test verifies `@Singleton` behavior.

src/test/groovy/example/micronaut/SingletonScopeSpec.groovy

Copy

```groovy
package example.micronaut


import io.micronaut.core.type.Argument
import io.micronaut.http.HttpRequest
import io.micronaut.http.client.HttpClient
import io.micronaut.http.client.annotation.Client
import io.micronaut.test.extensions.spock.annotation.MicronautTest
import jakarta.inject.Inject
import spock.lang.Specification
import spock.lang.Unroll

@MicronautTest (1)
class SingletonScopeSpec extends Specification {
    @Inject
    @Client('/')
    HttpClient httpClient (2)

    @Unroll

    void 'only one instance of the bean exists for singleton beans'() {
        given:
        Set<String> responses = executeRequest(httpClient, path) as Set

        expect:
        responses.size() == 1 (3)

        when:
        responses.addAll(executeRequest(httpClient, path))

        then:
        responses.size() == 1 (4)

        where:
        path << ['/singleton']
    }

    private static List<String> executeRequest(HttpClient client, String path) {
        client.toBlocking().retrieve(HttpRequest.GET(path), Argument.listOf(String))
    }
}
```

(1) Annotate the class with `@MicronautTest` so the Micronaut framework will initialize the application context and the embedded server. More info.

(2) Inject the `HttpClient` bean and point it to the embedded server.

(3) The same instance fulfills both injection points at `RobotFather` and `RobotMother`.

(4) Same instance is used upon subsequent requests.

7. Prototype

Prototype scope indicates that a new instance of the bean is created each time it is injected

Let's use `@Prototype` instead of `@Singleton`.

src/main/groovy/example/micronaut/prototype/Robot.groovy

Copy

```
package example.micronaut.prototype

import io.micronaut.context.annotation.Prototype

import java.util.UUID

@Prototype (1)
class Robot {
    private final String serialNumber = UUID.randomUUID().toString()

    String getSerialNumber() {
        serialNumber
    }
}
```

(1) Use `io.micronaut.context.annotation.Prototype` to designate the scope of bean as Prototype - a non-singleton scope that creates a new bean for every injection point.

7.1. Prototype Tests

The following test verifies the behavior of `Prototype` scope.

src/test/groovy/example/micronaut/PrototypeScopeSpec.groovy

Copy

```
package example.micronaut


import io.micronaut.core.type.Argument
import io.micronaut.http.HttpRequest
import io.micronaut.http.client.HttpClient
import io.micronaut.http.client.annotation.Client
import io.micronaut.test.extensions.spock.annotation.MicronautTest
import jakarta.inject.Inject
import spock.lang.Specification
import spock.lang.Unroll

@MicronautTest (1)
class PrototypeScopeSpec extends Specification {
    @Inject
    @Client('/')
    HttpClient httpClient (2)

    @Unroll

    void 'prototype scope indicates that a new instance of the bean is created each time it is injected'() {
        given:
        Set<String> responses = executeRequest(httpClient, path) as Set

        expect:
        responses.size() == 2 (3)

        when:
        responses.addAll(executeRequest(httpClient, path))

        then:
        responses.size() == 2 (4)

        where:
        path << ['/prototype']
    }

    private static List<String> executeRequest(HttpClient client, String path) {
        client.toBlocking().retrieve(HttpRequest.GET(path), Argument.listOf(String))
    }
}
```

(1) Annotate the class with `@MicronautTest` so the Micronaut framework will initialize the application context and the embedded server. More info.

(2) Inject the `HttpClient` bean and point it to the embedded server.

(3) A new instance is created to fulfill each injection point. Two instances - one for `RobotFather` and another for `RobotMother`.

(4) Instances remain upon subsequent requests.

8. Request

@RequestScope scope is a custom scope that indicates a new instance of the bean is created and associated with each HTTP request

src/main/groovy/example/micronaut/request/Robot.groovy

Copy

```groovy
package example.micronaut.request

import io.micronaut.http.HttpRequest
import io.micronaut.runtime.http.scope.RequestAware
import io.micronaut.runtime.http.scope.RequestScope

@RequestScope (1)
class Robot implements RequestAware { (2)
    String serialNumber

    @Override
    void setRequest(HttpRequest<?> request) {
        serialNumber = request.headers.get('UUID')
    }
}
```

1. Use `io.micronaut.runtime.http.scope.RequestScope` to designate the scope of bean as Request - a new instance of the bean is created and associated with each HTTP request.
2. `RequestAware` API allows `@RequestScope` beans to access to the current request.

8.1. Request Tests

The following test verifies the behavior of `@RequestScope` scope.

src/test/groovy/example/micronaut/RequestScopeSpec.groovy

Copy

```groovy
package example.micronaut


import io.micronaut.core.type.Argument
import io.micronaut.http.HttpRequest
import io.micronaut.http.client.HttpClient
import io.micronaut.http.client.annotation.Client
import io.micronaut.test.extensions.spock.annotation.MicronautTest
import jakarta.inject.Inject
import spock.lang.Specification

@MicronautTest (1)
class RequestScopeSpec extends Specification {
    @Inject
    @Client('/')
    HttpClient httpClient (2)


    void 'request scope creates an instance associated with each HTTP request'() {
        given:
        String path = '/request'
        Set<String> responses = executeRequest(httpClient, createRequest(path)) as Set

        expect:
        responses.size() == 1 (3)

        when:
        responses.addAll(executeRequest(httpClient, createRequest(path)))

        then:
        responses.size() == 2 (4)
    }

    private static List<String> executeRequest(HttpClient client, HttpRequest<?> request) {
        client.toBlocking().retrieve(request, Argument.listOf(String))
    }

    private static HttpRequest<?> createRequest(String path) {
        HttpRequest.GET(path).header('UUID', UUID.randomUUID().toString())
    }
}
```

1. Annotate the class with `@MicronautTest` so the Micronaut framework will initialize the application context and the embedded server. More info.
2. Inject the `HttpClient` bean and point it to the embedded server.

3. Both injection points, `RobotFather` and `RobotMother`, are fulfilled with the same instance of `Robot`. An instance associated with the HTTP Request.
4. Both injection points are fulfilled with the new instance of `Robot`.

9. Refreshable

Refreshable scope is a custom scope that allows a bean's state to be refreshed via the /refresh endpoint.

src/main/groovy/example/micronaut/refreshable/Robot.groovy

```
package example.micronaut.refreshable

import io.micronaut.runtime.context.scope.Refreshable

import java.util.UUID

@Refreshable (1)
class Robot {
    private final String serialNumber = UUID.randomUUID().toString()

    String getSerialNumber() {
        serialNumber
    }
}
```

Copy

1. `@Refreshable` scope is a custom scope that allows a bean's state to be refreshed via the /refresh endpoint.

Your application needs the `management` dependency to enable the refresh endpoint.

build.gradle

```
implementation("io.micronaut:micronaut-management")
```

Copy

9.1. Refreshable Tests

The following test enables the **refresh endpoint** and verifies the behavior of `@Refreshable`

src/test/groovy/example/micronaut/RefreshableScopeSpec.groovy

Copy

```groovy
package example.micronaut


import io.micronaut.context.annotation.Property
import io.micronaut.core.type.Argument
import io.micronaut.core.util.StringUtils
import io.micronaut.http.HttpRequest
import io.micronaut.http.client.HttpClient
import io.micronaut.http.client.annotation.Client
import io.micronaut.test.extensions.spock.annotation.MicronautTest
import jakarta.inject.Inject
import spock.lang.Specification

@Property(name = 'endpoints.refresh.enabled', value = StringUtils.TRUE) (1)
@Property(name = 'endpoints.refresh.sensitive', value = StringUtils.FALSE) (2)
@MicronautTest (3)
class RefreshableScopeSpec extends Specification {
    @Inject
    @Client('/')
    HttpClient httpClient (4)


    void 'refreshable scope allows a bean state to be refreshed via the refresh endpoint'() {
        given:
        String path = '/refreshable'
        Set<String> responses = executeRequest(httpClient, path) as Set

        expect:
        responses.size() == 1 (5)

        when:
        responses.addAll(executeRequest(httpClient, path))

        then:
        responses.size() == 1 (6)

        when:
        refresh(httpClient) (7)
        responses.addAll(executeRequest(httpClient, path))

        then:
        responses.size() == 2 (8)
    }

    private static void refresh(HttpClient client) {
        client.toBlocking().exchange(HttpRequest.POST('/refresh', [force: true]))
    }

    private static List<String> executeRequest(HttpClient client, String path) {
        client.toBlocking().retrieve(HttpRequest.GET(path), Argument.listOf(String))
    }
}
```

(1) Annotate the class with `@Property` to supply configuration to the test.
(2) The refresh endpoint is sensitive by default. To invoke it in the test, we set `endpoints.refresh.sensitive` to false.
(3) Annotate the class with `@MicronautTest` so the Micronaut framework will initialize the application context and the embedded server. More info.
(4) Inject the `HttpClient` bean and point it to the embedded server.
(5) The same instance fulfills both injection points at `RobotFather` and `RobotMother`.
(6) The same instance serves a new request.
(7) Hitting the refresh endpoint, publishes a RefreshEvent, which invalidates the instance of `Robot`.
(8) A new instance of `Robot` is created the next time the object is requested.

10. @Context

Context scope indicates that the bean will be created at the same time as the ApplicationContext (eager initialization)

The following example uses `@Context` in combination with `@ConfigurationProperties`.

src/main/groovy/example/micronaut/context/FrameworkConfiguration.groovy

Copy

```
package example.micronaut.context

import io.micronaut.context.annotation.ConfigurationProperties
import io.micronaut.context.annotation.Context
import jakarta.validation.constraints.Pattern

@Context (1)
@ConfigurationProperties('framework') (2)
class FrameworkConfiguration {

    @Pattern(regexp = 'groovy|java|kotlin') (3)
    String language
}
```

1. The life cycle of classes annotated with `io.micronaut.context.annotation.Context` is bound to that of the bean context.
2. The `@ConfigurationProperties` annotation takes the configuration prefix.
3. Use `jakarta.validation.constraints` Constraints to ensure the data matches your expectations.

The result is validation being performed on the Application Context start-up.

src/test/groovy/example/micronaut/ContextSpec.groovy

```
package example.micronaut

import io.micronaut.context.ApplicationContext
import io.micronaut.context.exceptions.BeanInstantiationException
import spock.lang.Specification

class ContextSpec extends Specification {

    void 'life cycle of classes annotated with Context is bound to that of the BeanContext'() {
        when:
        ApplicationContext.run(['framework.language': 'scala'])

        then:
        BeanInstantiationException e = thrown()
        e.message.contains('must match "groovy|java|kotlin"')
    }
}
```

Copy

11. Other scopes

Micronaut Framework ships with other built-in Scopes:

11.1. @Infrastructure

@Infrastructure scope represents a bean that cannot be overridden or replaced using `@Replaces` because it is critical to the functioning of the system.

11.2. @ThreadLocal

@ThreadLocal scope is a custom scope that associates a bean per thread via a ThreadLocal

11.3. Next Steps

Read more about **Scopes** in the Micronaut Framework.

12. Help with the Micronaut Framework

The **Micronaut Foundation** sponsored the creation of this Guide. A variety of **consulting and support services** are available.

13. License

All guides are released with an Apache license 2.0 license for the code and a Creative Commons Attribution 4.0 license for the writing and media (images...).

MICRONAUT FOUNDATION™

BRAND GUIDELINES
COMMUNITY GUIDELINES
PRIVACY POLICY